Geth Import 및 Export 명령어 상세 분석

`geth import` 와 `geth export` 명령어는 블록체인 데이터를 파일로 내보내거나 파일에서 가져오는 핵심 명령어입니다.

> **명령어:** `geth import <file> / geth export <file>`

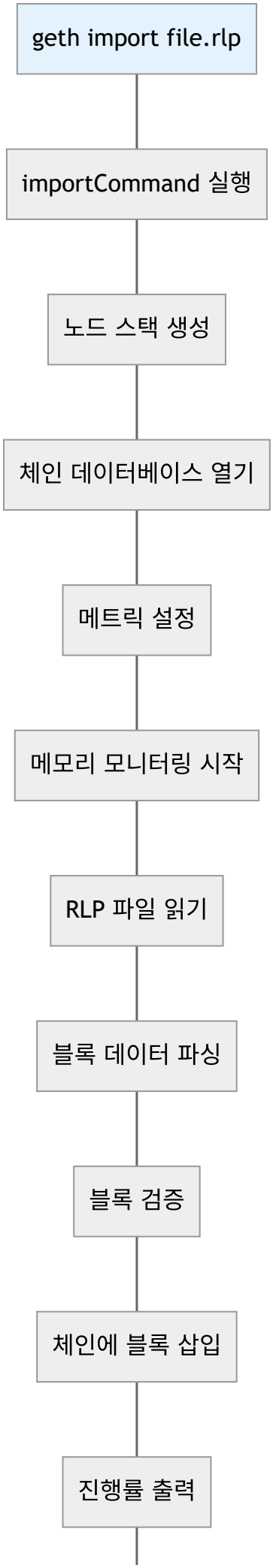
> **진입점:** `importChain() / exportChain()` 함수

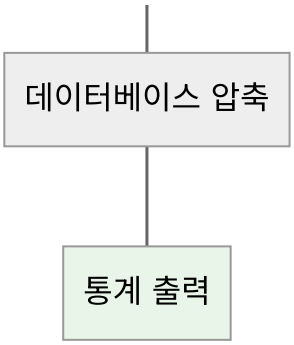
> **소스 파일:** `cmd/geth/chaincmd.go`

> **카테고리:** 체인 관리

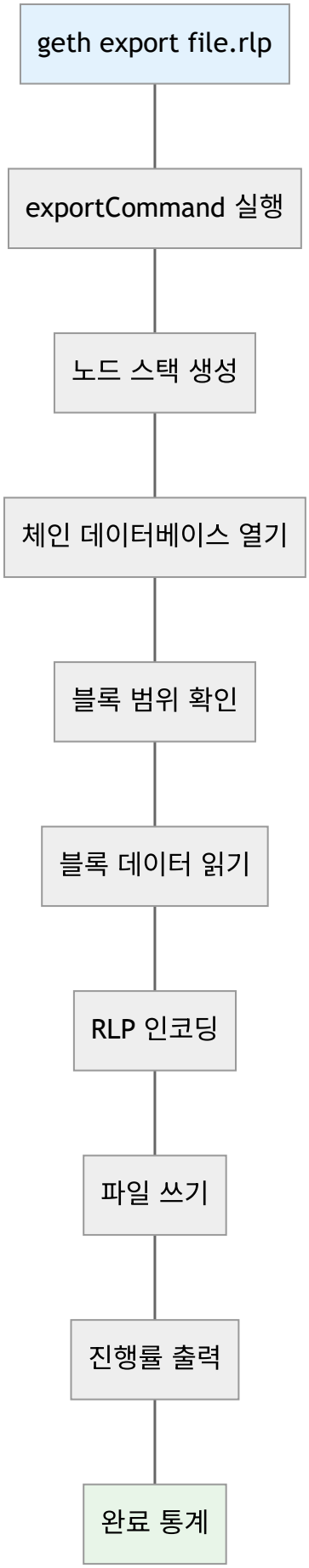
🚀 전체 실행 흐름 개요

Import 흐름


geth import file.rlp
importCommand 실행
노드 스택 생성
체인 데이터베이스 열기
메트릭 설정
메모리 모니터링 시작
RLP 파일 읽기
블록 데이터 파싱
블록 검증
체인에 블록 삽입
진행률 출력



Export 흐름



📋 Import 명령어 상세 분석

1단계: 명령어 정의

```
// cmd/geth/chaincmd.go
importCommand = &cli.Command{
Action: importChain,
Name: "import",
Usage: "Import a blockchain file",
ArgsUsage: "<filename> (<filename 2> ... <filename N>) ",
Flags: slices.Concat([]cli.Flag{
utils.GCModeFlag,
utils.SnapshotFlag,
utils.CacheFlag,
utils.CacheDatabaseFlag,
utils.CacheTrieFlag,
utils.CacheGCFlag,
utils.CacheSnapshotFlag,
utils.CacheNoPrefetchFlag,
utils.CachePreimagesFlag,
utils.NoCompactionFlag,
utils.MetricsEnabledFlag,
// ... 더 많은 플래그들
utils.TxLookupLimitFlag,
utils.VMTraceFlag,
```

```
utils.TransactionHistoryFlag,

utils.StateHistoryFlag,

}, utils.DatabaseFlags, debug.Flags),

Description: `

The import command allows the import of blocks from an RLP-encoded format. This

containing multiple RLP-encoded blocks, or multiple files can be given.


If only one file is used, an import error will result in the entire import proce

multiple files are processed, the import process will continue even if an indivi

to import successfully.`,

}
```

2단계: Import 실행 함수 (`importChain()`)

```
// importChain는 RLP 인코딩된 블록체인 파일을 가져옵니다.

func  importChain(ctx *cli.Context) error {

// 1. 인수 검증

if  ctx.Args().Len() < 1 {

utils.Fatalf("This command requires an argument.")

}


// 2. 노드 스택 및 설정 생성

stack, cfg := makeConfigNode(ctx)
```

```
defer stack.Close()

// 3. 메트릭 설정 (활성화된 경우)
utils.SetupMetrics(&cfg.Metrics)

// 4. 체인 및 데이터베이스 생성
chain, db := utils.MakeChain(ctx, stack, false) // readonly = false
defer db.Close()

// 5. 메모리 사용량 모니터링 시작
var peakMemAlloc, peakMemSys atomic.Uint64
go func() {
stats := new(runtime.MemStats)
for {
runtime.ReadMemStats(stats)
if peakMemAlloc.Load() < stats.Alloc {
peakMemAlloc.Store(stats.Alloc)
}
if peakMemSys.Load() < stats.Sys {
peakMemSys.Store(stats.Sys)
}
time.Sleep(5 * time.Second)
}
```

```
}()

// 6. 실제 Import 실행
start := time.Now()
var importErr error

if ctx.Args().Len() == 1 {
// 단일 파일 Import
if err := utils.ImportChain(chain, ctx.Args().First()); err != nil {
importErr = err
log.Error("Import error", "err", err)
}
} else {
// 다중 파일 Import
for _, arg := range ctx.Args().Slice() {
if err := utils.ImportChain(chain, arg); err != nil {
importErr = err
log.Error("Import error", "file", arg, "err", err)
if err == utils.ErrImportInterrupted {
break // 중단 신호 시 전체 중단
}
}
}
```

```
}

// 7. 체인 중지 및 통계 출력
chain.Stop()
fmt.Printf("Import done in %v.\n\n", time.Since(start))

// 8. 데이터베이스 통계 출력
showDBStats(db)

// 9. 메모리 사용량 통계 출력
mem := new(runtime.MemStats)
runtime.ReadMemStats(mem)

fmt.Printf("Object memory: %.3f MB current, %.3f MB peak\n",
float64(mem.Alloc)/1024/1024,
float64(peakMemAlloc.Load())/1024/1024)
fmt.Printf("System memory: %.3f MB current, %.3f MB peak\n",
float64(mem.Sys)/1024/1024,
float64(peakMemSys.Load())/1024/1024)
fmt.Printf("Allocations: %.3f million\n", float64(mem.Mallocs)/1000000)
fmt.Printf("GC pause: %v\n\n", time.Duration(mem.PauseTotalNs))

// 10. 데이터베이스 압축 (옵션)
```

```
if !ctx.Bool(utils.NoCompactionFlag.Name) {
start = time.Now()
fmt.Println("Compacting entire database...")
if err := db.Compact(nil, nil); err != nil {
utils.Fatalf("Compaction failed: %v", err)
}
fmt.Printf("Compaction done in %v.\n\n", time.Since(start))
showDBStats(db)
}

return importErr
}
```

3단계: 실제 Import 로직 (`utils.ImportChain()`)

```
// cmd/utils/cmd.go의 ImportChain 함수
func ImportChain(chain *core.BlockChain, fn string) error {
// 1. 파일 열기
fmt.Printf("Importing blockchain from %s\n", fn)
fh, err := os.Open(fn)
if err != nil {
return err
}
```

```
defer fh.Close()

// 2. gzip 압축 파일 처리
var reader io.Reader = fh
if strings.HasSuffix(fn, ".gz") {
if reader, err = gzip.NewReader(reader); err != nil {
return err
}
}

// 3. RLP 스트림 생성
stream := rlp.NewStream(reader, 0)

// 4. 블록 배치 처리 설정
blocks := make(types.Blocks, 0, 2500) // 배치 크기: 2500 블록

// 5. 진행률 표시 설정
var (
imported = 0
start = time.Now()
reported = time.Now()
)
```

```
// 6. 블록 읽기 및 처리 루프
for {
// 7. RLP에서 블록 디코딩
var b types.Block
if err := stream.Decode(&b); err == io.EOF {
break // 파일 끝
} else if err != nil {
return fmt.Errorf("at block %d: %v", imported, err)
}

// 8. 블록을 배치에 추가
blocks = append(blocks, &b)

// 9. 배치가 가득 찼거나 마지막 블록인 경우 처리
if len(blocks) == cap(blocks) {
if err := importBlocks(chain, blocks); err != nil {
return fmt.Errorf("batch %d-%d: %v",
imported, imported+len(blocks)-1, err)
}
imported += len(blocks)
blocks = blocks[:0]

// 10. 진행률 출력 (5초마다)
```

```
if time.Since(reported) >= 5*time.Second {
fmt.Printf("Imported %d blocks, elapsed %v\n",
imported, time.Since(start))
reported = time.Now()
}
}

// 11. 중단 신호 확인
select {
case <-interrupt:
return ErrImportInterrupted
default:
}
}

// 12. 남은 블록들 처리
if len(blocks) > 0 {
if err := importBlocks(chain, blocks); err != nil {
return fmt.Errorf("final batch: %v", err)
}
imported += len(blocks)
}
```

```
fmt.Printf("Imported %d blocks\n", imported)
return nil
}

// importBlocks는 블록 배치를 체인에 삽입합니다.
func importBlocks(chain *core.BlockChain, blocks types.Blocks) error {
// 1. 블록 검증 및 삽입
index, err := chain.InsertChain(blocks)
if err != nil {
if index < len(blocks) {
log.Error("Block import failed",
"block", blocks[index].Number(),
"hash", blocks[index].Hash(),
"err", err)
}
return err
}
return nil
}
```

📋 Export 명령어 상세 분석

1단계: 명령어 정의

```
// cmd/geth/chaincmd.go

exportCommand = &cli.Command{

Action: exportChain,

Name: "export",

Usage: "Export blockchain into file",

ArgsUsage: "<filename> [<blockNumFirst> <blockNumLast>]",

Flags: slices.Concat([]cli.Flag{utils.CacheFlag}, utils.DatabaseFlags),

Description: `

Requires a first argument of the file to write to.

Optional second and third arguments control the first and

last block to write. In this mode, the file will be appended

if already existing. If the file ends with .gz, the output will

be gzipped.`,

}
```

2단계: Export 실행 함수 (`exportChain()`)

```
// exportChain는 블록체인을 RLP 형식으로 파일에 내보냅니다.

func  exportChain(ctx *cli.Context) error {

// 1. 인수 검증

if  ctx.Args().Len() < 1 {

utils.Fatalf("This command requires an argument.")

}
```

```go
// 2. 노드 스택 생성
stack, _ := makeConfigNode(ctx)
defer stack.Close()

// 3. 체인 및 데이터베이스 생성 (읽기 전용)
chain, db := utils.MakeChain(ctx, stack, true) // readonly = true
defer db.Close()

start := time.Now()
var err error
fp := ctx.Args().First()

// 4. 블록 범위 확인
if ctx.Args().Len() < 3 {
// 전체 체인 내보내기
err = utils.ExportChain(chain, fp)
} else {
// 특정 범위 내보내기
first, ferr := strconv.ParseInt(ctx.Args().Get(1), 10, 64)
last, lerr := strconv.ParseInt(ctx.Args().Get(2), 10, 64)

if ferr != nil || lerr != nil {
```

```
utils.Fatalf("Export error in parsing parameters: block number not an integer\n"
}
if first < 0 || last < 0 {
utils.Fatalf("Export error: block number must be greater than 0\n")
}

// 5. 블록 번호 유효성 검사
if head := chain.CurrentSnapBlock(); uint64(last) > head.Number.Uint64() {
utils.Fatalf("Export error: block number %d larger than head block %d\n",
uint64(last), head.Number.Uint64())
}

err = utils.ExportAppendChain(chain, fp, uint64(first), uint64(last))
}

if err != nil {
utils.Fatalf("Export error: %v\n", err)
}

fmt.Printf("Export done in %v\n", time.Since(start))
return nil
}
```

3단계: 실제 Export 로직 (`utils.ExportChain()`)

```
// cmd/utils/cmd.go의 ExportChain 함수
func ExportChain(blockchain *core.BlockChain, fn string) error {
fmt.Printf("Exporting blockchain to %s\n", fn)

// 1. 전체 체인 범위 설정
head := blockchain.CurrentSnapBlock()
return ExportAppendChain(blockchain, fn, 0, head.Number.Uint64())
}

// ExportAppendChain는 지정된 범위의 블록을 파일에 내보냅니다.
func ExportAppendChain(blockchain *core.BlockChain, fn string, first, last ui
fmt.Printf("Exporting blocks [%d, %d] to %s\n", first, last, fn)

// 1. 파일 생성/열기
fh, err := os.OpenFile(fn, os.O_CREATE|os.O_WRONLY|os.O_APPEND, os.ModePerm)
if err != nil {
return err
}
defer fh.Close()

// 2. gzip 압축 설정 (파일명이 .gz로 끝나는 경우)
```

```
var writer io.Writer = fh
if strings.HasSuffix(fn, ".gz") {
writer = gzip.NewWriter(writer)
defer writer.(*gzip.Writer).Close()
}

// 3. 진행률 표시 설정
var (
exported = uint64(0)
start = time.Now()
reported = time.Now()
)

// 4. 블록 내보내기 루프
for nr := first; nr <= last; nr++ {
// 5. 블록 가져오기
block := blockchain.GetBlockByNumber(nr)
if block == nil {
return fmt.Errorf("block %d not found", nr)
}

// 6. RLP 인코딩 및 쓰기
if err := rlp.Encode(writer, block); err != nil {
```

```
return err
}
exported++

// 7. 진행률 출력 (5초마다)
if time.Since(reported) >= 5*time.Second {
fmt.Printf("Exported %d blocks, elapsed %v\n",
exported, time.Since(start))
reported = time.Now()
}

// 8. 중단 신호 확인
select {
case <-interrupt:
return ErrExportInterrupted
default:
}
}

fmt.Printf("Exported %d blocks\n", exported)
return nil
}
```

🔧 History Import 및 Export (Era Archives)

Import History 명령어

```
// importHistoryCommand는 Era 아카이브를 가져옵니다.
importHistoryCommand = &cli.Command{
Action: importHistory,
Name: "import-history",
Usage: "Import an Era archive",
ArgsUsage: "<dir>",
Flags: slices.Concat([]cli.Flag{
utils.TxLookupLimitFlag,
utils.TransactionHistoryFlag
}, utils.DatabaseFlags, utils.NetworkFlags),
Description: `
The import-history command will import blocks and their corresponding receipts
from Era archives.
`,
}
```

```
// importHistory 함수
func  importHistory(ctx *cli.Context) error {
if  ctx.Args().Len() != 1 {
```

```
utils.Fatalf("usage: %s", ctx.Command.ArgsUsage)

}


stack, _ := makeConfigNode(ctx)

defer stack.Close()


chain, db := utils.MakeChain(ctx, stack, false)

defer db.Close()


var (

start = time.Now()

dir = ctx.Args().Get(0)

network string

)


// 1. 네트워크 결정

if utils.IsNetworkPreset(ctx) {

switch {

case ctx.Bool(utils.MainnetFlag.Name):

network = "mainnet"

case ctx.Bool(utils.SepoliaFlag.Name):

network = "sepolia"

case ctx.Bool(utils.HoleskyFlag.Name):
```

```
network = "holesky"

}

} else {

// 디렉토리에서 네트워크 자동 감지

var  networks []string

for  _, n := range  params.NetworkNames {

entries, err := era.ReadDir(dir, n)

if  err != nil {

return  fmt.Errorf("error reading %s: %w", dir, err)

}

if  len(entries) > 0 {

networks = append(networks, n)

}

}



if  len(networks) == 0 {

return  fmt.Errorf("no era1 files found in %s", dir)

}

if  len(networks) > 1 {

return  errors.New("multiple networks found, use a network flag to specify desir

}

network = networks[0]

}
```

```
// 2. Era 파일 읽기 및 Import
entries, err := era.ReadDir(dir, network)
if err != nil {
return fmt.Errorf("error reading era dir: %w", err)
}

for _, entry := range entries {
if err := utils.ImportHistory(chain, db, entry, dir, network); err != nil {
return err
}
}

fmt.Printf("Import history done in %v\n", time.Since(start))
return nil
}
```

Export History 명령어

```
// exportHistoryCommand는 블록체인을 Era 아카이브로 내보냅니다.
exportHistoryCommand = &cli.Command{
Action: exportHistory,
Name: "export-history",
```

```
Usage: "Export blockchain history to Era archives",
ArgsUsage: "<dir> <first> <last>",
Flags: utils.DatabaseFlags,
Description: `
The export-history command will export blocks and their corresponding receipts
into Era archives. Eras are typically packaged in steps of 8192 blocks.
`,
}

// exportHistory 함수
func exportHistory(ctx *cli.Context) error {
if ctx.Args().Len() != 3 {
utils.Fatalf("usage: %s", ctx.Command.ArgsUsage)
}

stack, _ := makeConfigNode(ctx)
defer stack.Close()

chain, db := utils.MakeChain(ctx, stack, true)
defer db.Close()

var (
dir = ctx.Args().Get(0)
```

```
first = ctx.Args().Get(1)
last = ctx.Args().Get(2)
start = time.Now()
)

// 1. 블록 번호 파싱
firstNum, err := strconv.ParseUint(first, 10, 64)
if err != nil {
return fmt.Errorf("invalid first block number: %v", err)
}

lastNum, err := strconv.ParseUint(last, 10, 64)
if err != nil {
return fmt.Errorf("invalid last block number: %v", err)
}

// 2. Era 아카이브 생성
if err := utils.ExportHistory(chain, db, dir, firstNum, lastNum); err != nil {
return err
}

fmt.Printf("Export history done in %v\n", time.Since(start))
return nil
```

```
}
```

🔧 주요 구성 요소

사용 가능한 플래그

Import 플래그

| 플래그 | 설명 | 기본값 |

| ---------- | ---------- | ---------- |

| `--cache` | 메모리 캐시 크기 (MB) | 1024 |

| `--cache.database` | 데이터베이스 캐시 비율 | 50% |

| `--cache.trie` | 트라이 캐시 크기 (MB) | 15% of cache |

| `--cache.gc` | GC 캐시 크기 (MB) | 25% of cache |

| `--cache.snapshot` | 스냅샷 캐시 크기 (MB) | 10% of cache |

| `--no-compaction` | Import 후 압축 건너뛰기 | false |

| `--txlookuplimit` | 트랜잭션 인덱스 제한 | 2350000 |

| `--state.history` | 상태 히스토리 보관 | 90000 |

Export 플래그

| 플래그 | 설명 | 기본값 |

| ---------- | ---------- | ---------- |

| `--cache` | 메모리 캐시 크기 (MB) | 1024 |

| `--datadir` | 데이터 디렉토리 | `~/.ethereum` |

RLP 파일 형식

RLP (Recursive Length Prefix) 인코딩:

- Ethereum에서 사용하는 직렬화 형식
- 블록, 트랜잭션, 영수증 등을 바이너리로 인코딩
- 압축 효율성과 파싱 속도 최적화

파일 구조:

```
[Block1_RLP][Block2_RLP][Block3_RLP]...[BlockN_RLP]
```

각 블록은 다음 구조로 인코딩됩니다:

```
type Block struct {
Header *Header
Transactions []*Transaction
Uncles []*Header
}
```

📊 실행 예시

1. 전체 체인 내보내기

```
$ geth export blockchain.rlp
Exporting blockchain to blockchain.rlp
Exported 18500000 blocks
Export done in 2h15m30s
```

2. 특정 범위 내보내기

```
$ geth export blocks_1000_2000.rlp 1000 2000
Exporting blocks [1000, 2000] to blocks_1000_2000.rlp
Exported 1001 blocks
Export done in 45s
```

3. 압축된 파일로 내보내기

```
$ geth export blockchain.rlp.gz
Exporting blockchain to blockchain.rlp.gz
Exported 18500000 blocks
Export done in 1h45m20s
```

4. 블록체인 가져오기

```
$ geth import blockchain.rlp
Importing blockchain from blockchain.rlp
Imported 2500 blocks, elapsed 15s
Imported 5000 blocks, elapsed 30s
...
Imported 18500000 blocks
Import done in 3h20m15s
```

```
Object memory: 2048.000 MB current, 4096.000 MB peak
System memory: 8192.000 MB current, 12288.000 MB peak
Allocations: 1250.000 million
GC pause: 2m30s

Compacting entire database...
Compaction done in 45m20s

Database stats:
Size: 750.5 GB
Keys: 125,000,000
Reads: 2,500,000,000
Writes: 1,250,000,000
```

5. 다중 파일 가져오기

```
$ geth import file1.rlp file2.rlp file3.rlp
Importing blockchain from file1.rlp
Imported 1000 blocks
Importing blockchain from file2.rlp
Imported 1000 blocks
Importing blockchain from file3.rlp
```

```
Imported 1000 blocks

Import done in 2m30s
```

6. Era 히스토리 가져오기

```
$ geth import-history --mainnet ./era_archives/

Importing Era archives from ./era_archives/

Processing era-00000000-00008191.era1

Processing era-00008192-00016383.era1

...

Import history done in 4h15m30s
```

🛠️ 성능 최적화

1. 메모리 설정 최적화

```
# 대용량 Import를 위한 메모리 설정

$ geth import --cache 8192 --cache.database 75 blockchain.rlp


# 트라이 캐시 증가

$ geth import --cache 4096 --cache.trie 1024 blockchain.rlp


# 스냅샷 캐시 증가 (빠른 동기화)
```

```
$ geth import --cache 4096 --cache.snapshot 1024 blockchain.rlp
```

2. 디스크 I/O 최적화

```
# 압축 건너뛰기 (빠른 Import)
$ geth import --no-compaction blockchain.rlp

# SSD에 임시 데이터 저장
$ geth import --datadir /ssd/ethereum blockchain.rlp
```

3. 네트워크별 최적화

```
# 메인넷 (대용량)
$ geth import --cache 16384 --cache.database 80 mainnet.rlp

# 테스트넷 (소용량)
$ geth import --cache 2048 testnet.rlp
```

🔍 문제 해결

일반적인 오류들

1. 메모리 부족 오류

```
Fatal: Import error: runtime: out of memory
```

해결: 캐시 크기 줄이기 또는 시스템 메모리 증가

```
$ geth import --cache 1024 blockchain.rlp
```

2. 디스크 공간 부족

```
Import error: write /home/user/.ethereum/geth/chaindata: no space left on
```

해결: 디스크 공간 확보 또는 다른 위치에 데이터 디렉토리 설정

```
$ geth import --datadir /external/ethereum blockchain.rlp
```

3. 손상된 RLP 파일

```
Import error: at block 12345: rlp: input contains more than one value
```

해결: 파일 무결성 확인, 다시 다운로드

4. 블록 검증 실패

```
Import error: invalid block: invalid merkle root
```

해결: 올바른 네트워크용 파일인지 확인, 제네시스 블록 일치 여부 확인

성능 문제 해결

1. Import 속도가 느림

- 캐시 크기 증가: `--cache 8192`
- SSD 사용: 데이터 디렉토리를 SSD로 이동
- 압축 건너뛰기: `--no-compaction`

2. 메모리 사용량이 높음

- 캐시 크기 감소: `--cache 1024`
- GC 빈도 증가: `GOGC=50 geth import ...`

3. Export 속도가 느림

- 읽기 전용 모드 확인 (자동으로 설정됨)
- 압축 사용: `.gz` 확장자로 파일 크기 감소

📈 고급 사용법

1. 스크립트를 통한 자동화

```
#!/bin/bash
# blockchain_backup.sh

DATE=$(date +%Y%m%d)
BACKUP_DIR="/backup/ethereum"
LATEST_BLOCK=$(geth --exec "eth.blockNumber" attach)

# 최신 1000 블록 백업
START_BLOCK=$((LATEST_BLOCK - 1000))
geth export "$BACKUP_DIR/blocks_${START_BLOCK}_${LATEST_BLOCK}_${DATE}.rlp.gz"
```

```
echo "백업 완료: blocks_${START_BLOCK}_${LATEST_BLOCK}_${DATE}.rlp.gz"
```

2. 증분 백업

```
#!/bin/bash
# incremental_backup.sh

LAST_BACKUP_BLOCK=$(cat /backup/last_block.txt 2>/dev/null || echo "0")
CURRENT_BLOCK=$(geth --exec "eth.blockNumber" attach)

if [ $CURRENT_BLOCK -gt $LAST_BACKUP_BLOCK ]; then
NEXT_BLOCK=$((LAST_BACKUP_BLOCK + 1))
geth export "/backup/incremental_${NEXT_BLOCK}_${CURRENT_BLOCK}.rlp.gz" $NEXT
echo $CURRENT_BLOCK > /backup/last_block.txt
echo "증분 백업 완료: $NEXT_BLOCK ~ $CURRENT_BLOCK"
else
echo "새로운 블록이 없습니다."
fi
```

3. 네트워크 간 데이터 이전

```
# 메인넷에서 Export
geth  --mainnet  export  mainnet_blocks.rlp.gz  0  1000000


# 프라이빗 네트워크로 Import (제네시스 블록 호환성 주의)
geth  --datadir  ./private_net  import  mainnet_blocks.rlp.gz
```

4. 선택적 데이터 Import

```
# 트랜잭션 인덱스 제한 (공간 절약)
geth  import  --txlookuplimit  100000  blockchain.rlp


# 상태 히스토리 제한
geth  import  --state.history  50000  blockchain.rlp


# 스냅샷 비활성화
geth  import  --snapshot=false  blockchain.rlp
```

참고 자료:

- Geth Import/Export 가이드
- RLP 인코딩 명세
- Era 아카이브 형식

이 문서는 go-ethereum 소스 코드 분석을 바탕으로 작성되었습니다.